UNIT-4

Regression analysis, Regression: linear regression simple linear regression, multiple & Polynomial regression, Sparse model. Unsupervised learning, clustering, similarity and distances, quality measures of clustering, case study.

Regression Analysis

Introduction

In this chapter, we introduce *regression analysis* and some of its applications in data science. Regression is related to how to make predictions about real-world quantities such as, for instance, the predictions alluded to in the following questions. How does sales volume change with changes in price? How is sales volume affected by the weather? How does the title of a book affect its sales? How does the amount of a drug absorbed vary with the patient's body weight; and does this relationship depend on blood pressure? How many customers can I expect today? At what time should Igo home to avoid traffic jams? What is the chance of rain on the next two Mondays; and what is the expected temperature?

All these questions have a common structure: they ask for a *response* that canbe expressed as a combination of one or more (independent) *variables* (also called*covariates* or *predictors*). The role of regression is to build a model to predict theresponse from the variables. This process involves the transition *from data to model*.More specifically, the model can be useful in different tasks, such as the following:

(1) analyzing the behavior of data (the relation between the response and the vari- ables), (2) predicting data values (whether continuous or discrete), and (3) finding important variables for the model.

In order to understand how a regression model can be suitable for tackling these tasks, we will introduce three practical cases for which we use three real datasets and solve different questions. These practical cases will motivate *simple linear regression*, *multiple linear regression*, and *logistic regression*, as presented in the following sections.

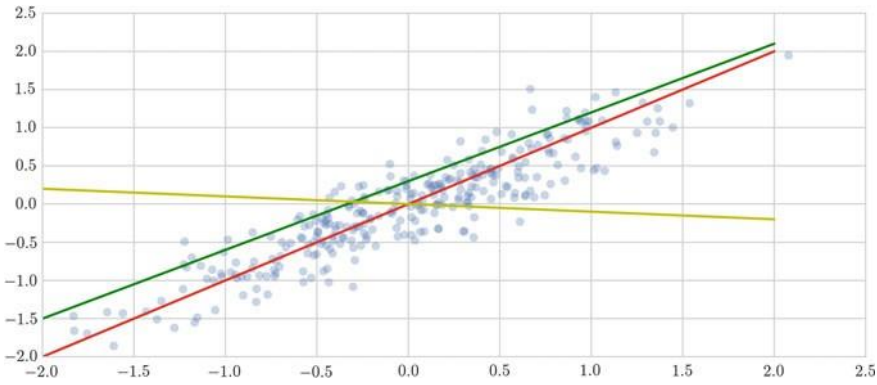


Fig. 6.1 Illustration of different simple linear regression models. *Blue points* correspond to a set of random points sampled from a univariate normal (Gaussian) distribution. *Red*, *green* and *yellow lines* are three different simple linear regression models

Linear Regression

The objective of performing a regression is to build a model to express the relation between the response $\mathbf{y} \in \mathrm{R}^n$ and a combination of one or more (independent) variables $\mathbf{x}_i \in \mathrm{R}^n$. [1] The model allows us to predict the response $\mathbf{y}$ from the variables. The simplest model which can be considered is a *linear model*, where the response $\mathbf{y}$ depends linearly on the d variables $\mathbf{x}_i$:

$$\mathbf{y} = a_1\mathbf{x}_1 + \cdots + a_d\mathbf{x}_d. \tag{6.1}$$

The variables a_i are termed the *parameters* or *coefficients* of the model. This equation can be rewritten in a more compact matrix form: $\mathbf{y} = \mathbf{X}\mathbf{w}$, where

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \mathbf{X} = \begin{pmatrix} x_{11} \ldots x_{1d} \\ x_{21} \ldots x_{2d} \\ \vdots \\ x_{n1} \ldots x_{nd} \end{pmatrix}, \mathbf{w} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_d \end{pmatrix}.$$

Linear regression is the technique for creating these linear models.

Simple Linear Regression

Simple linear regression considers n samples of a single variable $\mathbf{x} \in \mathrm{R}^n$ and describes the relationship between the variable and the response with the model:

$$\mathbf{y} = a_0 + a_1\mathbf{x}, \tag{6.2}$$

where the parameter a_0 is called the *intercept* or the *constant term*.

Given a set of samples $(\mathbf{x}, \mathbf{y})$, such as the set illustrated in Fig. 6.1, we can create a linear model to explain the data, as in Eq. (6.2). But how do we know which is the

best model (best parameters) for this particular set of samples? See the three different models (straight lines in different colors) in Fig. 6.1.

Ordinary least squares (OLS) is the simplest and most common *estimator* in which the parameters (a's) are chosen to minimize the square of the distance between thepredicted values and the actual values with respect to a_0, a_1:

$$||a_0 + a_1\mathbf{x} - \mathbf{y}||^2 = \sum_{j=1}^{n} (a_0 + a_1 x_j - y_j)^2.$$

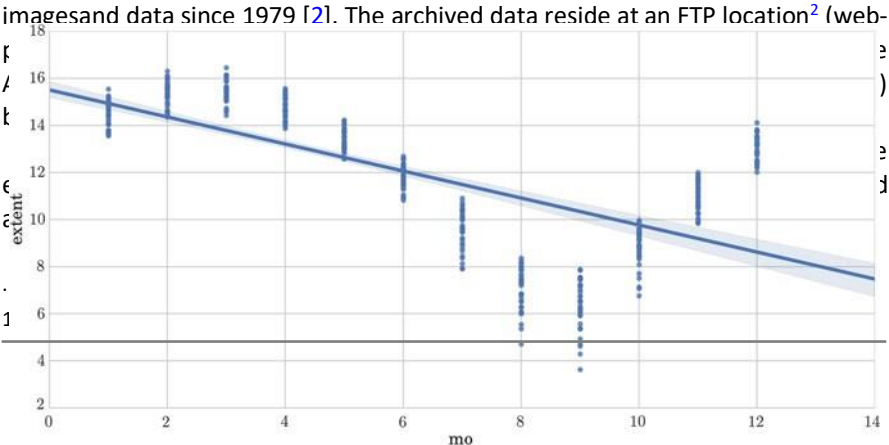
We are concerned here with the y-axis distance, since it does not consider the error in the variables. This error expression is often called the *sum of squared errors of prediction* (SSE). The SSE function is quadratic in the parameters, **w**, with positive-definite Hessian, and therefore this function possesses a unique global minimum at $\mathbf{w} \quad (a_0, a_1)$. The resulting model is represented as follows: $\mathbf{y} \quad a_0 \; a_1\mathbf{x}$, where the hats on the variables represent the fact that they are estimated from the data available.

OLS is a popular approach for several reasons. It makes it computationally cheap to calculate the coefficients. It is also easier to interpret than the other more sophisticated models. In situations where the goal is to understand a simple model in detail, ratherthan to estimate the response well, it can provide insight into what the model captures. Finally, in situations where there is a lot of noise, as in many real scenarios, it maybe hard to find the true functional form, so a constrained model can perform quite well compared to a complex model which can be more affected by noise.

Practical Case: Sea Ice Data and Climate Change

In this practical case, we pose the question: Is the climate really changing? More concretely, we want to show the effect of the climate change by determining whether the *sea ice area* (or *extent*) has decreased over the years. Sea ice area refers to the total area covered by ice, whereas sea ice extent is the area of ocean with at least 15% sea ice. Reliable measurement of sea ice edges began with the satellite era in the late 1970s. Before then, sea ice area and extent were monitored less precisely bya combination of ships, buoys, and aircraft.

We will use the sea ice data from the National Snow & Ice Data Center[1] which provides measurements of the area and extend of sea ice at the poles over the last 36 years. The center has given access to the archived monthly Sea Ice Index imagesand data since 1979 [2]. The archived data reside at an FTP location[2] (web-



0 ession Analysis
0

Fig. 6.2 Ice extent data by month

6

In[1]:

R
e
g

First, we read the data, previously downloaded, and create a DataFrame (*Pandas*) as follows:

```
ice = pd.read_csv('files/ch06/SeaIce.txt',
                  delim_whitespace=True)
print 'shape:', ice.shape
```

Out[1]: shape: (424, 6)

For data cleaning, we check the values of all the fields to detect any potential error. We find that there is a ' 9999' value in the data_type field which should contain 'Goddard' or 'NRTSI-G' (the type of the input dataset). So we can easily clean the data, removing these instances.

In[2]:

```
ice2 = ice[ice.data_type != '-9999']
```

Next, we visualize the data. The lmplot() function from the *Seaborn* toolbox is intended for exploring linear relationships of different forms in multidimensional datasets. For instance, we can illustrate the relationship between the month of the year (variable) and the extent(response) as follows:

In[3]:

```
import Seaborn as sns
sns.lmplot("mo", "extent", ice2)
```

This outputs Fig. 6.2. We can observe a monthly fluctuation of the sea ice extent,as would be expected for the different seasons of the year.

We should normalize the data before performing the regression analysis to avoid this fluctuation and be able to study the evolution of the extent over the years. To capture the variation for a given interval of time (month), we can compute the mean

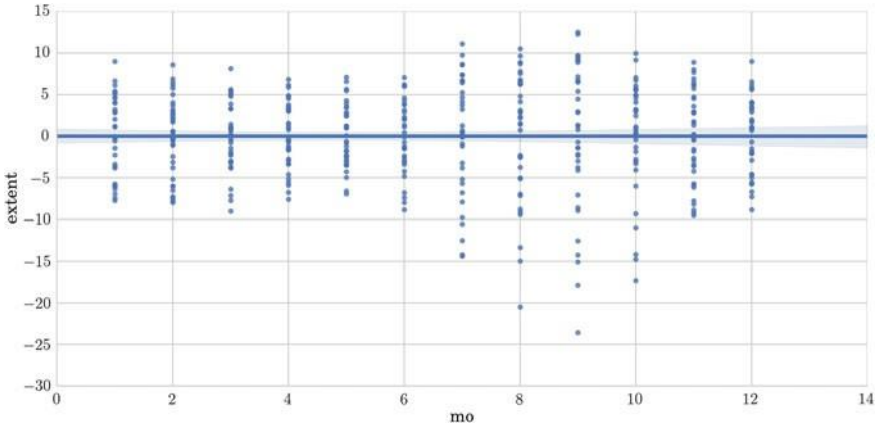


Fig. 6.3 Ice extent data by month after the normalization

for the i-th interval of time (using the period from 1979 through 2014 for the meanextent) μ_i , and subtract it from the set of extent values for that month $\{e_j^i\}$. This value can be converted to a relative percentage difference by dividing it by the totalaverage (1979–2014) μ, and then multiplying by 100:

$$\tilde{e}_j^i = 100 * \frac{e_j^i - \mu_i}{\mu}, i = 1, \ldots, 12.$$

We implement this normalization and plot the relationship again as follows:

In[4]:

```
for i in range(12):
    ice2.extent[ice2.mo == i+1] =
        100*(ice2.extent[ice2.mo == i+1]
            - month_means[i+1])
        /month_means.mean()
sns.lmplot("mo", "extent", ice2)
```

The new output is in Fig. 6.3. We now observe a comparable range of values forall months.

Next, the normalized values can be plotted for the entire time series to analyze the tendency. We compute the trend as a simple linear regression. We use the lmplot() function for visualizing linear relationships between the year (variable) and the extent (response).

In[5]:

```
sns.lmplot("year", "extent", ice2)
```

This outputs Fig. 6.4 showing the regression model fitting the extent data. This plot has two main components. The first is a scatter plot, showing the observed data points. The second is a regression line, showing the estimated linear model relating

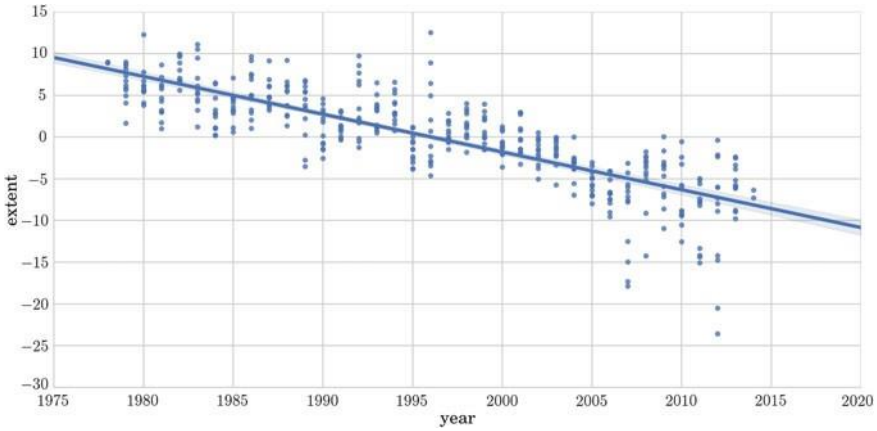


Fig. 6.4 Regression model fitting sea ice extent data for all months by year using lmplot

the two variables. The regression line is plotted with a 95% confidence band to givean impression of the uncertainty in the model.

In this figure, we can observe that the data show a long-term negative trend overyears. The negative trend can be attributed to global warming, although there is alsoa considerable amount of variation from year to year.

Up until here, we have qualitatively shown the linear regression using a useful visu- alization tool. We can also analyze the linear relationship in the data using the *Scikit- learn* library, which allows a quantitative evaluation. As was explained in the previous chapter, Scikit-learn provides an object-oriented interface centered around the con-cept of an estimator. The sklearn.linear_model.LinearRegression estimator sets the state of the estimator based on the training data using the function fit. Moreover, it allows the user to specify whether to fit an intercept term in the object construction. This is done by setting the corresponding constructor argumentsof the estimator object as follows:

In[6]:
```
from sklearn . linear_model import LinearRegression
est = LinearRegression(fit_intercept = True)
```

During the fitting process, the state of the estimator is stored in instance attributes that have a trailing underscore ('_'). For example, the coefficients of a LinearRegression estimator are stored in the attribute coef_. We fit a regres- sion model using years as variables (**x**) and the extent values as the response (**y**).

In[7]:
```
x = ice2[['year']]
y = ice2[['extent']]
est.fit(x, y)
print "Coefficients:", est.coef_
print "Intercept:", est.intercept_
```

Out[7]: Coefficients: [[-0.45275459]]
Intercept: [903.71640207]

Estimators that can generate predictions provide an Estimator.predict method. In the case of regression, Estimator.predict will return the predicted regression values. We can evaluate the model fitting by computing the mean squared error (MSE) and the coefficient of determination (R^2) of the model. The coefficient R^2 is defined as $(1 - u/v)$, with $u = (y - y)^2$ and $v = (y - y)^2$, where y is the mean. The best possible score for R^2 is 1.0, lower values are worse (it can also be negative). These measures can provide a quantitative answer to the question we are facing: Is there a negative trend in the evolution of sea ice extent over recent years? We can perform this analysis for a particular month or for all months together, as done in the following lines:

In [8]:
```
from sklearn import metrics
y_hat = est.predict(x)
print "MSE:", metrics.mean_squared_error(y_hat, y)
print "R^2:", metrics.r2_score(y_hat, y)
print 'var:', y.var()
```

Out[8]: MSE: 10.5391316398
R^2: 0.50678703821
var: 31.98324

The negative trend seen in Fig. 6.4 is validated by the MSE value which is small, 0.1%, and the R^2 value which is acceptable, given the variance of the data, 0.3%.

Given the model, we can also predict the extent value for the coming years. For instance, the predicted extent for January 2025 can be computed as follows:

In [9]:
```
x = [2025]
y_hat = model.predict(x)
m = 1 # January
y_hat = (y_hat*month_means.mean()/100) + month_means[m]
print "Prediction of extent for January 2025
      (in millions of square km):", y_hat
```

Out[9]: Prediction of extent for January 2025 (in millions of squarekm): [12.93603933].

Multiple Linear Regression and Polynomial Regression

As we have seen in the previous section, with simple linear regression we describe the relationship between the variable and the response with a straight line. In the case of *multiple linear regression*, we extend this idea by fitting a d-dimensional hyperplane to our d variables, as defined in Eq. (6.1).

Multiple linear regression may seem a very simple model, but even when the response depends on the variables in nonlinear ways, this model can still be used by

considering nonlinear transformations $\varphi(\cdot)$ of the variables:

$$\mathbf{y} = a_1\varphi(\mathbf{x}_1) + \cdots + a_d\varphi(\mathbf{x}_d)$$

This model is called *polynomial regression* and it is a popular nonlinear regression technique which models the relationship between the response and the variablesas an *p*-th order polynomial. The higher the order of the polynomial, the more complex the functions you can fit. However, using higher-order polynomial can involve *computational complexity* and *overfitting*. Overfitting occurs when a model fits the characteristics of the training data and loses the capacity to generalize fromthe seen to predict the unseen.

Sparse Model

Often, in real problems, there are uninformative variables in the data which prevent proper modeling of the problem and thus, the building of a correct regression model. In such cases, a feature selection process is crucial to select only the informative features and discard non-informative ones. This can be achieved by *sparse methods* which use a penalization approach, such as *LASSO* (least absolute shrinkage and selection operator) to set some model coefficients to zero (thereby discarding thosevariables). Sparsity can be seen as an application of Occam's razor: prefer simpler models to complex ones.

Given the set of samples $(\mathbf{X}, \mathbf{y})$, the objective of a sparse model is to minimize the SSE through a restriction (or penalty):

$$\frac{1}{2n}||\mathbf{Xw} - \mathbf{y}||_2^2 + \alpha||\mathbf{w}||_1,$$

where $||\mathbf{w}||_1$ is the $L1$-norm of the parameter vector $\mathbf{w} = (a_0, \ldots, a_d)$.

Practical Case: Prediction of the Price of a New Housing Market

In this practical case we want to solve the question: Can we predict the price of a new market given any of its attributes?

We will use the Boston housing dataset from Scikit-learn, which provides recorded measurements of 13 attributes of housing markets around Boston, as well as the median house price.[3] Once we load the dataset (506 instances), the description of the dataset can easily be shown by printing the field DESCR. The data (**x**), feature names, and target (**y**) are stored in other fields of the dataset.

We first consider the task of predicting median house values in the Boston area using as the variable one of the attributes, for instance, LSTAT, defined as the "pro-portion of lower status of the population".

Seaborn visualization can be used to show this linear relationships easily:

[3]Copy of UCI ML housing dataset: http://archive.ics.uci.edu/ml/datasets/Housing.

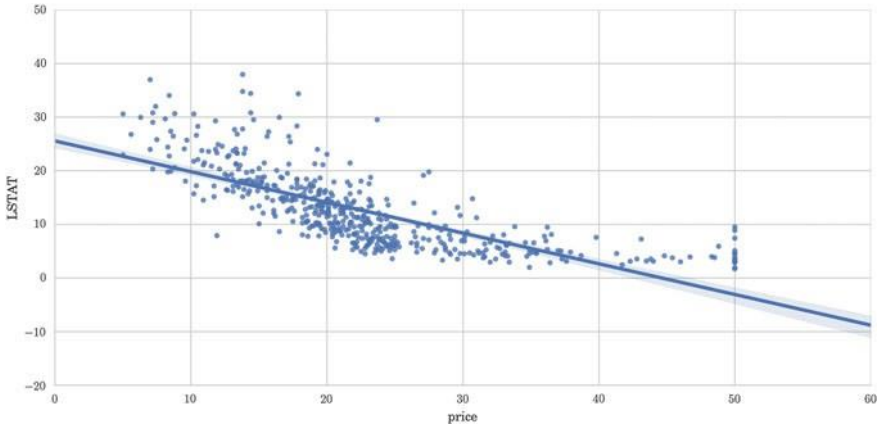


Fig. 6.5 Scatter plot of Boston data (LSTATversus price) and their linear relationship (using lmplot)

In [10]:
```
from sklearn import datasets
boston = datasets.load_boston()
X_boston, y_boston = boston.data, boston.target
print 'Shape of data:', X_boston.shape, y_boston.shape
print 'Feature names:',boston.feature_names
df_boston = pd.DataFrame(boston.data,
                         columns = boston.feature_names)
df_boston['price'] = boston.target
sns.lmplot("price", "LSTAT", df_boston)
```

Out[10]:Shape of data: (506L, 13L) (506L,)
Feature names: ['CRIM' 'ZN' 'INDUS' 'CHAS' 'NOX' 'RM' 'AGE'
'DIS' 'RAD' 'TAX' 'PTRATIO' 'B' 'LSTAT']

In Fig. 6.5, we can clearly see that the relationship between price and LSTAT is nonlinear, since the straight line is a poor fit. We can examine whether a better fit can be obtained by including higher-order terms. For example, a quadratic model:

$$\mathbf{y}_i \approx a_0 + a_1\mathbf{x}_i + a_2\mathbf{x}^2{}_i$$

The lmplotfunction allows to easily change the order of the model as is done in the next code, which outputs Fig. 6.6, where we observe a better fit.

In [11]:
```
sns.lmplot("price", "LSTAT", df_boston, order = 2)
```

To study the relation among multiple variables in a dataset, there are different options. We can study the relationship between several variables in a dataset by using the functions corr and heatmap which allow to calculate a correlation matrix for a dataset and draws a *heat map* with the correlation values. The heat mapis a matricial image which helps to interpret the correlations among variables. For the sake of visualization, we do not consider all the 13 variables in the Boston housing data, but six: CRIM, per capita crime rate by town; INDUS, proportion of non-retail

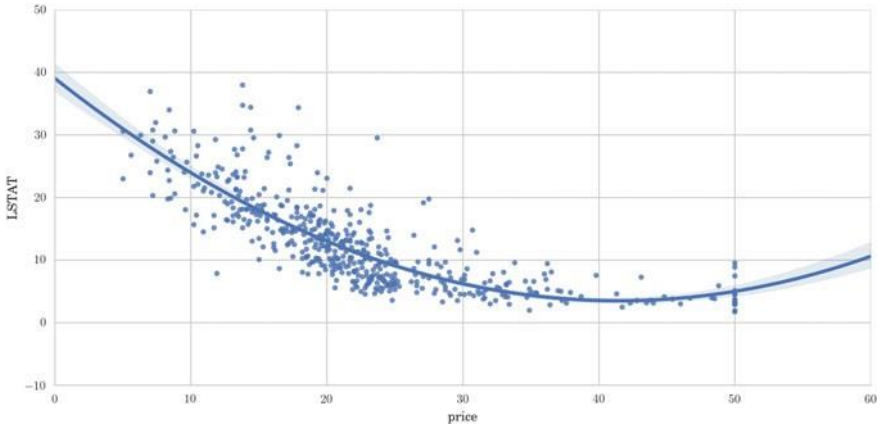


Fig. 6.6 Scatter plot of Boston data (LSTAT versus price) and their polynomial relationship(using lmplotwith order 2)

business acres per town; NOX, nitric oxide concentrations (parts per 10 million); RM,average number of rooms per dwelling; AGE, proportion of owner-occupied units built prior to 1940; and LSTAT. These variables are indicated by their indexes in thefollowing code:

In[12]:
```
indexes = [0,2,4,5,6,12]
df2 = pd.DataFrame(boston.data[:,indexes],
                   columns = boston.feature_names[indexes])
df2['price'] = boston.target
corrmat = df2.corr()
sns.heatmap(corrmat, vmax = .8, square = True)
```

Figure 6.7 shows a heat map representing the correlation between pairs of variables; specifically, the six variables selected and the price of houses. The color bar shows the range of values used in the matrix. This plot is a useful way of summa- rizing the correlation of several variables. It can be seen that LSTAT and RM are the variables that are most correlated with price.

Another good way to explore multiple variables is the *scatter plot* from Pandas. The scatter plot is a grid of plots of multiple variables one against the others, illus-trating the relationship of each variable with the rest. For the sake of visualization,we do not consider all the variables, but just three: RM, AGE, and LSTAT defined by indexesin the following code:

In[13]:
```
indexes =[5,6,12]
df2 = pd.DataFrame(boston.data[:,indexes],
                   columns = boston.feature_names[indexes])
df2['price'] = boston.target
pd.scatter_matrix(df2, figsize = (12.0, 12.0))
```

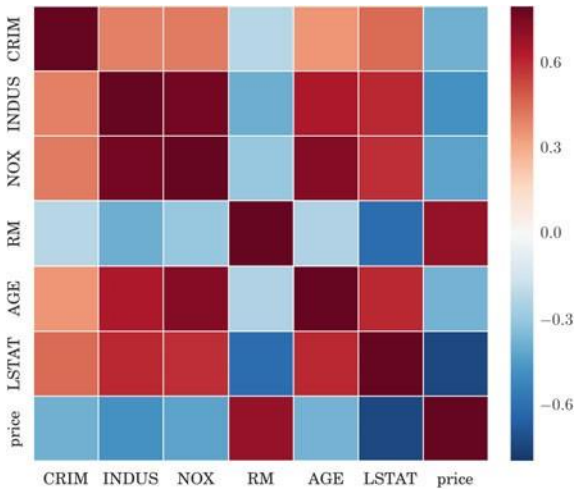


Fig. 6.7 Correlation plot: heat map representing thecorrelation between sevenpairs of variables in the Boston housing dataset

This code outputs Fig. 6.8, where we obtain visual information concerning the density function for every variable, in the diagonal, as well as the scatter plots of thedata points for pairs of variables. In the last column, we can appreciate the relationbetween the three variables selected and house prices. It can be seen that RMfollowsa linear relation with price; whereas AGE does not. LSTAT follows a higher-order relation with price. This plot gives us an indication of how good or bad every attribute would be as a variable in a linear model.

For the evaluation of the prediction power of the model with new samples, we split the data into a training set and a testing set, and we compute the linear regression score, which returns the coefficient of determination R^2 of the prediction. We can also calculate the MSE.

In [14]:
```
from sklearn import linear_model
train_size = X_boston.shape[0]/2
X_train = X_boston[:train_size]
X_test = X_boston[train_size:]
y_train = y_boston[:train_size]
y_test = y_boston[train_size:]
print 'Training and testing set sizes',
      X_train.shape, X_test.shape
regr = LinearRegression()
regr.fit(X_train, y_train)
print 'Coeff and intercept:',
      regr.coef_, regr.intercept_
print 'Testing Score:', regr.score(X_test, y_test) print '
    Training
MSE: ',
      np.mean((regr.predict(X_train) - y_train)**2)
print 'Testing MSE: ',
      np.mean((regr.predict(X_test) - y_test)**2)
```

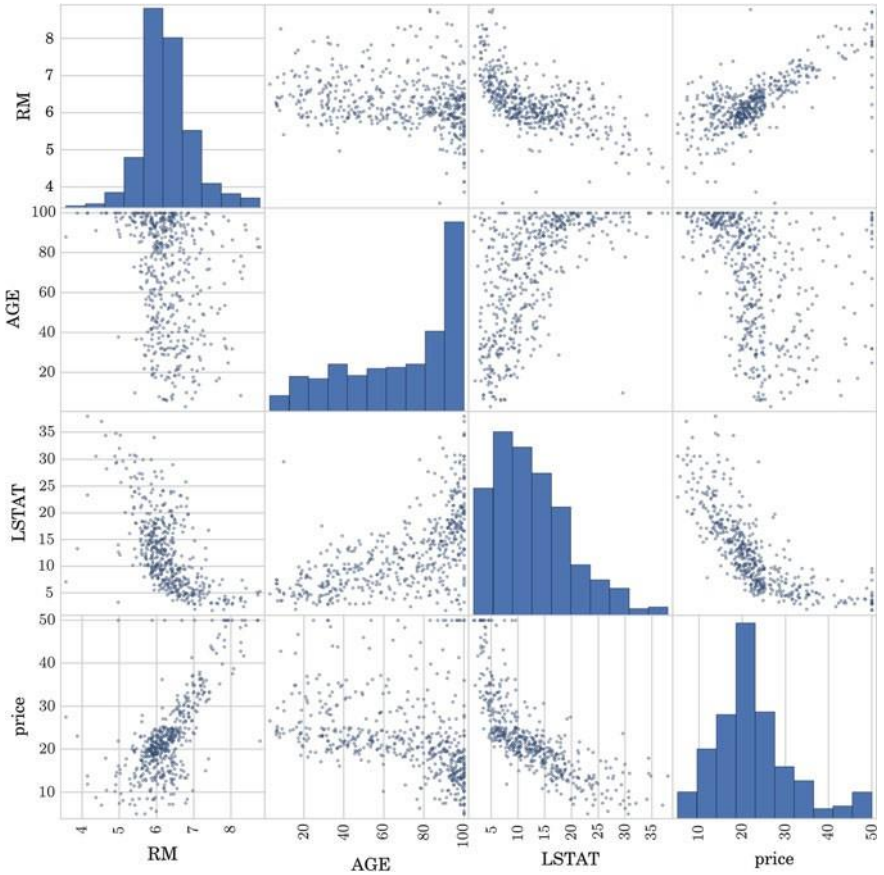


Fig. 6.8 Scatter plot of Boston housing dataset

Out[14]:Training and testing set sizes (253, 13) (253, 13)
Coeff and intercept: [1.20133313 0.02449686 0.00999508
0.42548672 -8.44272332 8.87767164 -0.04850422 -1.11980855
0.20377571 -0.01597724 -0.65974775 0.01777057 -0.11480104]
-10.0174305829
Testing Score: -2.24420202674
Training MSE: 9.98751732546
Testing MSE: 302.64091133

We can see that all the coefficients obtained are different from zero, meaning that no variable is discarded. Next, we try to build a sparse model to predict the price using the most important factors and discarding the non-informative ones. To do this, we can create a LASSO regressor, forcing zero coefficients.

In [15]:
```
regr_lasso = linear_model.Lasso(alpha = .3)
regr_lasso.fit(X_train, y_train) print 'Coeff and intercept:
    ',regr_lasso.coef_
print 'Tesing Score:', regr_lasso.score(X_test,
y_test) print 'Training MSE: ',
      np.mean((regr_lasso.predict(X_train) - y_train)**2)
print 'Testing MSE: ',
      np.mean((regr_lasso.predict(X_test) - y_test)**2)
```

Out[15]:
```
Coeff and intercept: [ 0. 0.01996512 -0. 0. -0. 7.69894744
-0.03444803 -0.79380636 0.0735163 -0.0143421 -0.66768539
0.01547437 -0.22181817] -6.18324183615
Testing Score: 0.501127529021
Training MSE: 10.7343110095
Testing MSE: 46.5381680949
```

It can now be seen that the result of the model fitting for a set of sparse coefficientsis much better than before (using all the variables), with the score increasing from
−2.24 to 0.5. This demonstrates that four of the initial variables are not importantfor the prediction and in fact they confuse the regressor.

With the LASSO result, we can also emphasize the most important factors for determining the price of a new market, based on the coefficient values:

In [16]:
```
ind = np.argsort(np.abs(regr_lasso.coef_))
print 'Ordered variable (from less to more important):',
      boston.feature_names[ind]
```

Out[16]:
```
Ordered variable (from less to more important): ['CRIM' 'INDUS' 'CHAS' 'NOX' 'TAX' 'B' 'ZN' 'AGE'
'RAD' 'LSTAT' 'PTRATIO' 'DIS''RM']
```

There are also other strategies for feature selection. For instance, we can select the k 5 best features, according to the k highest scores, using the function SelectKBestfrom Scikit-learn:

In [17]:
```
import sklearn.feature_selection as fs
selector = fs.SelectKBest(score_func = fs.f_regression,
                          k = 5)
selector.fit_transform(X_train, y_train) per
selector.fit(X_train,y_train)
print 'Selected features:',
      zip(selector.get_support(), boston.feature_names)
```

Out[17]:
```
Selected features: [(False, 'CRIM'), (False, 'ZN'), (True,
'INDUS'), (False, 'CHAS'), (False, 'NOX'), (True, 'RM'), (True,
'AGE'), (False, 'DIS'), (False, 'RAD'), (False, 'TAX'), (True,'PTRATIO'), (False, 'B'), (True, 'LSTAT')]
```

The set of selected features is now different, since the criterion has changed. However, three of the most important features: RM, PTRATIO, and LSTAT.

In order to evaluate the prediction, it could be interesting to visualize the targetand predicted responses in a scatter plot, as it is done in the next code:

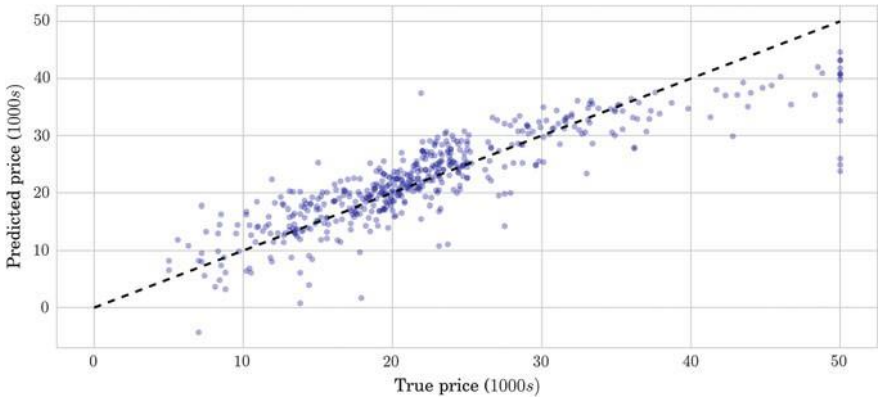


Fig. 6.9 Relation between true (x-axis) and predicted (y-axis) prices

In [18]:

```
clf = LinearRegression()
clf.fit(boston.data, boston.target)
predicted = clf.predict(boston.data)
plt.scatter(boston.target, predicted, alpha = 0.3)
plt.plot([0, 50], [0, 50], '--k')
plt.axis('tight')
plt.xlabel('True price ($1000s)')
plt.ylabel('Predicted price ($1000s)')
```

The output is shown in Fig. 6.9, where we can observe that the original prices are properly estimated by the predicted ones, except for the higher values, around
$50.000 (points in the top right corner).

Finally, it is worth noting that we can work with statistical evaluation of a linearregression with the *OLS* toolbox of the *Stats Model* toolbox.[4] This toolbox is usefulto study several statistics concerning the regression model. To know more about thetoolbox, go to the Documentation related to Stats Models.

Logistic Regression

Logistic regression is a type of model of probabilistic statistical classification. It is used as a binary model to predict a binary response, the outcome of a categorical dependent variable (i.e., a class label), based on one or more variables.

The form of the logistic function is:

$$f(x) = \frac{1}{1 + e^{-\lambda x}}$$

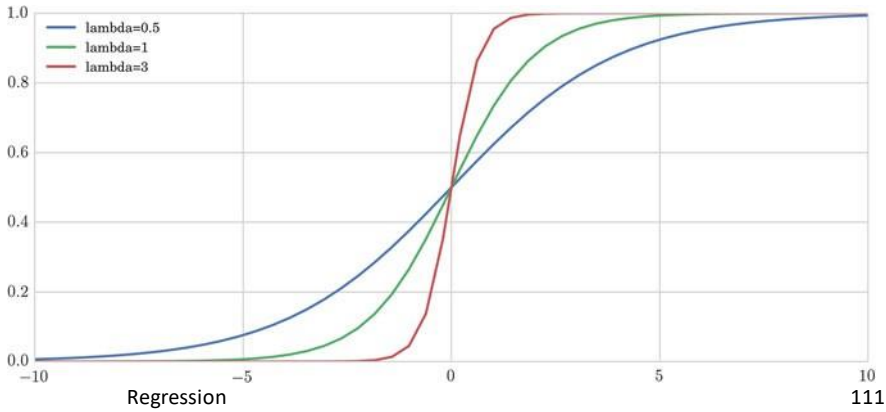


Fig. 6.10 Logistic function for different lambda values

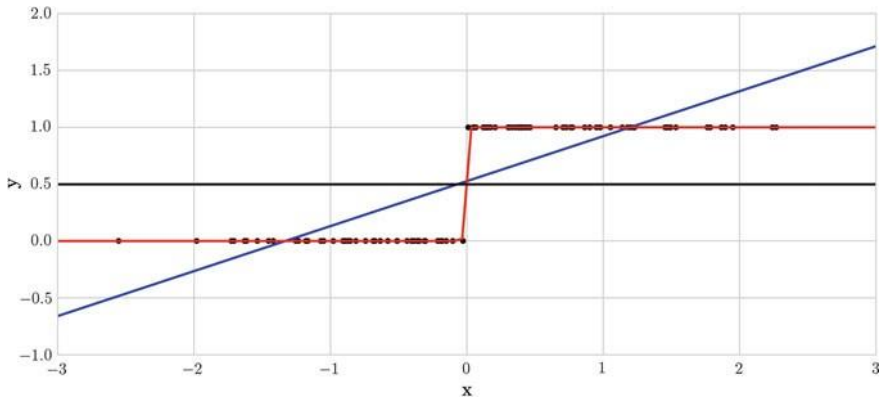


Fig. 6.11 Linear regression (*blue*) versus logistic regression (*red*) for fitting a set of data (*black points*) normally distributed across the 0 and 1 y-values

Figure 6.10 illustrates the logistic function with different values of λ. This function is useful because it can take as its input any value from negative infinity to positive infinity, whereas the output is restricted to values between 0 and 1 and hence can beinterpreted as a probability.

The set of samples (**X**, **y**), illustrated as black points in Fig. 6.11, defines a fitting problem suitable for a logistic regression. The blue and red lines show the fitting result for linear and logistic models, respectively. In this case, a logistic model can clearly explain the data; whereas a linear model cannot.

Practical Case: Winning or Losing Football Team

Now, we pose the question: What number of goals makes a football team the winner or the loser? More concretely, we want to predict victory or defeat in a football match when we are given the number of goals a team scores. To do this we consider

the set of results of the football matches from the Spanish league[5] and we build a classification model with it.

We first read the data file in a DataFrame and select the following columns in a new DataFrame: HomeTeam, AwayTeam, FTHG (home team goals), FTAG (away team goals), and FTR (H=home win, D draw, A away win). We then build a *d*-dimensional vector of variables with all the scores, **x**, and a binary response indicating victory or defeat, **y**. For that, we create two extra columns containing W the number of goals of the winning team and L the number of goals of the losing team and we concatenate these data. Finally, we can compute and visualize a logistic regression model to predict the discrete value (victory or defeat) using these data.

In [19]:

```
from sklearn.linear_model import LogisticRegression
data = pd.read_csv('files/ch06/SP1.csv')
s = data[['HomeTeam','AwayTeam', 'FTHG', 'FTAG', 'FTR']]
def my_f1(row):
    return max(row['FTHG'], row['FTAG'])
def my_f2(row):
    return min(row['FTHG'], row['FTAG'])
s['W'] = s.apply(my_f1, axis = 1)
s['L'] = s.apply(my_f2, axis = 1)
x1 = s['W'].values
y1 = np.ones(len(x1), dtype = np.int)
x2 = s['L'].values
y2 = np.zeros(len(x2), dtype = np.int)
x = np.concatenate([x1, x2])
x = x[:, np.newaxis]
y = np.concatenate([y1, y2])
logreg = LogisticRegression()
logreg.fit(x, y)
X_test = np.linspace(-5, 10, 300)
def lr_model(x):
    return 1 / (1+np.exp(-x))
loss = lr_model(X_test*logreg.coef_ + logreg.intercept_)
    .ravel()
X_test2 = X_test[:,np.newaxis]
losspred = logreg.predict(X_test2)
plt.scatter(x.ravel(), y,
            color = 'black',
            s = 100, zorder = 20,
            alpha = 0.03)
plt.plot(X_test, loss, color = 'blue', linewidth = 3)
plt.plot(X_test, losspred, color = 'red', linewidth = 3)
```

Figure 6.12 shows a scatter plot with transparency so we can appreciate the overlapping in the discrete positions of the total numbers of victories and defeats. It also shows the fitting of the logistic regression model, in blue, and prediction of the logistic regression model, in red, for the Spanish football league results. With this information we can estimate that the cutoff value is 1. This means that a team, in general, has to score more than one goal to win.

[5] http://www.football-data.co.uk/mmz4281/1213/SP1.csv.

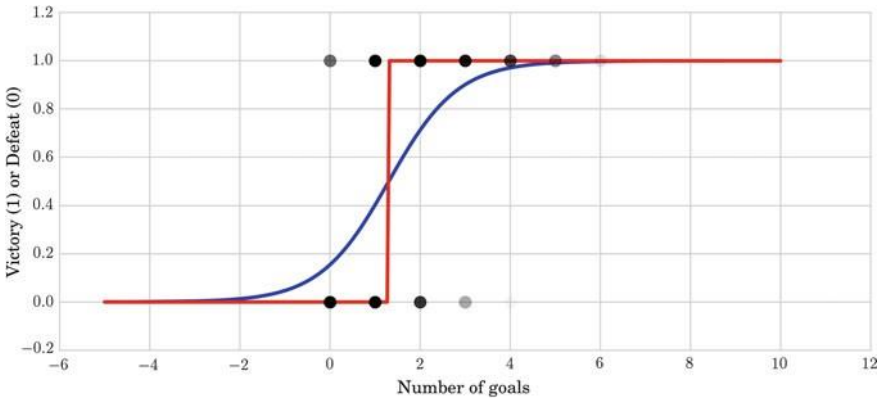


Fig. 6.12 Fitting of the logistic regression model (*blue*) and prediction of the logistic regression model (*red*) for the Spanish football league results

Unsupervised Learning

Introduction

In machine learning, the problem of unsupervised learning is that of trying to find hidden structure in unlabeled data. Since the examples given to the learner are unla- beled, there is no error or reward signal to evaluate the goodness of a potential solution. This distinguishes unsupervised from supervised learning. Unsupervised learning is defined as the task performed by algorithms that learn from a training setof unlabeled or unannotated examples, using the features of the inputs to categorizethem according to some geometric or statistical criteria.

Unsupervised learning encompasses many techniques that seek to summarize and explain key features or structures of the data. Many methods employed in unsuper- vised learning are based on data mining methods used to preprocess data. Most unsupervised learning techniques can be summarized as those that tackle the follow-ing four groups of problems:

- *Clustering*: has as a goal to partition the set of examples into groups.
- *Dimensionality reduction*: aims to reduce the dimensionality of the data. Here, weencounter techniques such as Principal Component Analysis (PCA), independentcomponent analysis, and nonnegative matrix factorization.
- *Outlier detection*: has as a purpose to find unusual events (e.g., a malfunction),that distinguish part of the data from the rest according to certain criteria.
- *Novelty detection*: deals with cases when changes occur in the data (e.g., in stream-ing data).

The most common unsupervised task is clustering, which we focus on in this chapter.

Clustering

Clustering is a process of grouping similar objects together; i.e., to partition unlabeled examples into disjoint subsets of clusters, such that:

- Examples within a cluster are similar (in this case, we speak of *high intraclass similarity*).
- Examples in different clusters are different (in this case, we speak of *low interclass similarity*).

When we denote data as similar and dissimilar, we should define a measure for this similarity/dissimilarity. Note that grouping similar data together can help in discov- ering new categories in an unsupervised manner, even when no sample category labels are provided. Moreover, two kinds of inputs can be used for grouping:

(a) in *similarity-based clustering*, the input to the algorithm is an $n \times n$ *dissimilarity matrix* or *distance matrix*;
(b) in *feature-based clustering*, the input to the algorithm is an $n \times D$ *feature matrix* or *design matrix*, where n is the number of examples in the dataset and D the dimensionality of each sample.

Similarity-based clustering allows easy inclusion of domain-specific similarity, while feature-based clustering has the advantage that it is applicable to potentially noisy data.

Therefore, several questions regarding the clustering process arise.

- What is a natural grouping among the objects? We need to define the “groupness” and the “similarity/distance” between data.
- How can we group samples? What are the best procedures? Are they efficient? Are they fast? Are they deterministic?
- How many clusters should we look for in the data? Shall we state this numbera priori? Should the process be completely data driven or can the user guide the grouping process? How can we avoid “trivial” clusters? Should we allow final clustering results to have very large or very small clusters? Which methods work when the number of samples is large? Which methods work when the number ofclasses is large?
- What constitutes a good grouping? What objective measures can be defined to evaluate the quality of the clusters?

There is not always a single or optimal answer to these questions. It used to be said that clustering is a “subjective” issue. Clustering will help us to describe, analyze, and gain insight into the data, but the quality of the partition depends to a great extent on the application and the analyst.

Similarity and Distances

To speak of similar and dissimilar data, we need to introduce a notion of the similarity of data. There are several ways for modeling of similarity. A simple way to model this is by means of a Gaussian kernel:

$$s(a, b) = e^{-\gamma d(a,b)}$$

where $d(a, b)$ is a metric function, and γ is a constant that controls the decay of the function. Observe that when $a=b$, the similarity is maximum and equal to one. On the contrary, when a is very different to b, the similarity tends to zero. The former modeling of the similarity function suggests that we can use the notion of distance as a surrogate. The most widespread distance metric is the *Minkowski distance*:

$$d(a, b) = \left(\sum_{i=1}^{d} |a_i - b_i|^p \right)^{1/p}$$

where $d(a, b)$ stands for the distance between two elements $a, b \in \mathrm{R}^d$, d is the dimensionality of the data, and p is a parameter.

The best-known instantiations of this metric are as follows:

- when $p = 2$, we have the *Euclidean distance*,
- when $p = 1$, we have the *Manhattan distance*, and
- when $p = \inf$, we have the *max-distance*. In this case, the distance corresponds tothe component $|a_i - b_i|$ with the highest value.

What Constitutes a Good Clustering? Defining Metricsto Measure Clustering Quality

When performing clustering, the question normally arises: How do we measure thequality of the clustering result? Note that in unsupervised clustering, we do not have groundtruth labels that would allow us to compute the accuracy of the algorithm. Still, there are several procedures for assessing quality. We find two families of techniques: those that allow us to compare clustering techniques, and those that check on specific properties of the clustering, for example “compactness”.

Rand Index, Homogeneity, Completeness and V-measureScores

One of the best-known methods for comparing the results in clustering techniques in statistics is the *Rand index* or Rand measure (named after William M. Rand). TheRand index evaluates the similarity between two results of data clustering. Since in unsupervised clustering, class labels are not known, we use the Rand index to compare the coincidence of different clusterings obtained by different approaches or criteria. As an alternative, we later discuss the

Silhouette coefficient: instead of

comparing different clusterings, this evaluates the compactness of the results of applying a specific clustering approach.

Given a set of n elements $S = \{o_1, \ldots, o_n\}$ we can compare two partitions of S[1]: $X = \{X_1, \ldots, X_r\}$, a partition of S into r subsets; and $Y = \{Y_1, \ldots, Y_s\}$, a partition of S into s subsets. Let us use the annotations as follows:

- a is the number of pairs of elements in S that are in the same subset in both X and Y;
- b is the number of pairs of elements in S that are in different subsets in both X and Y;
- c is the number of pairs of elements in S that are in the same subset in X, but indifferent subsets in Y; and
- d is the number of pairs of elements in S that are in different subsets in X, but inthe same subset in Y.

The Rand index, R, is defined as follows:

$$R = \frac{a+b}{a+b+c+d},$$

ensuring that its value is between 0 and 1.

One of the problems of the Rand index is that when given two datasets with random labelings, it does not take a constant value (e.g., zero) as expected. Moreover, when the number of clusters increases it is desirable that the upper limit tends to the unity. To solve this problem, a form of the Rand index, called the *Adjusted Rand index*, is used that adjusts the Rand index with respect to chance grouping of elements. It is defined as follows:

$$AR = \frac{\binom{n}{2}(a+d) - [(a+b)(a+c)+(c+d)(b+d)]}{\binom{n}{2}^2 - [(a+b)(a+c)+(c+d)(b+d)]}.$$

Another way for comparing clustering results is the V-measure. Let us first introduce some concepts. We say that a clustering result satisfies a *homogeneity* criterion if all of its clusters contain only data points which are members of the same original (single) class. A clustering result satisfies a *completeness* criterion if all the data points that are members of a given class are elements of the same predicted cluster. Note that both scores have real positive values between 0.0

In[1]:
```
print("%.3f" % metrics.homogeneity_score([0, 0, 1, 1],
                                         [0, 0, 0, 0]))
```

Out[1]: 0.000

.

The homogeneity is 0 since the samples in the predicted cluster 0 come fromoriginal cluster 0 and cluster 1.

In[2]:

```
print metrics.completeness_score([0, 0, 1, 1],
                                 [1, 1, 0, 0])
```

Out[2]: 1.0

The completeness is 1 since all the samples from the original cluster with label 0 go into the same predicted cluster with label 1, and all the samples from the originalcluster with label 1 go into the same predicted cluster with label 0.

However, how can we define a measure that takes into account the completenessas well as the homogeneity? The *V-measure* is the harmonic mean between the homogeneity and the completeness defined as follows:

$$v = 2 * (\text{homogeneity} * \text{completeness}) / (\text{homogeneity} + \text{completeness}).$$

Note that this metric is not dependent of the absolute values of the labels: a permutation of the class or cluster label values will not change the score value in any way. Moreover, the metric is symmetric with respect to switching between the predicted and the original cluster label. This is very useful to measure the agreement of two independent label assignment strategies applied to the same dataset even when the real groundtruth is not known. If class members are completely split across different clusters, the assignment is totally incomplete, hence the V-measure is null:

In [3]:
```
print("%.3f" % metrics.v_measure_score([0, 0, 0, 0],
                                        [0, 1, 2, 3]))
```

Out[3]: 0.000

In contrast, clusters that include samples from different classes destroy the homo-geneity of the labeling, hence:

In [4]:
```
print("%.3f" % metrics.v_measure_score([0, 0, 1, 1],
                                        [0, 0, 0, 0]))
```

Out[4]: 0.000

In summary, we can say that the advantages of the V-measure include that it has bounded scores: 0.0 means the clustering is extremely bad; 1.0 indicates a per-fect clustering result. Moreover, it can be interpreted easily: when analyzing the V-measure, low completeness or homogeneity explain in which direction the clus- tering is not performing well. Furthermore, we do not assume anything about the cluster structure. Therefore, it can be used to compare clustering algorithms suchas *K-means*, which assume isotropic blob shapes, with results of other clustering algorithms such as spectral clustering (see Sect. 7.2.3.2), which can find clusters with "folded" shapes. As a drawback, the previously introduced metrics are not normalized with regard to random labeling. This means that depending on the num- ber of samples, clusters and groundtruth classes, a completely random labeling will

not always yield the same values for homogeneity, completeness and hence, the

V-measure. In particular, random labeling will not yield a zero score, and they will tend further from zero as the number of clusters increases. It can be shown that this prob-lem can reliably be overcome when the number of samples is high, i.e., more than athousand, and the number of clusters is less than 10. These metrics require knowl- edge of the groundtruth classes, while in practice this information is almost never available or requires manual assignment by human annotators. Instead, as mentioned before, these metrics can be used to compare the results of different clusterings.

Silhouette Score

An alternative to the former scores is to evaluate the final 'shape' of the clustering result. This is the underlying idea behind the Silhouette coefficient. It is defined asa function of the intracluster distance of a sample in the dataset, *a* and the nearest- cluster distance, *b* for each sample.[2] Later, we will discuss different ways to compute the distance between clusters. The Silhouette coefficient for a sample *i* can be written as follows:

$$Silhouette(i) = \frac{b - a}{\max(a, b)}.$$

Hence, if the Silhouette *s(i)* is close to 0, it means that the sample is on the border ofits cluster and the closest one from the rest of the dataset clusters. A negative valuemeans that the sample is closer to the neighbor cluster. The average of the Silhouettecoefficients of all samples of a given cluster defines the "goodness" of the cluster. A high positive value, i.e., close to 1 would mean a compact cluster, and vice versa. And the average of the Silhouette coefficients of all clusters gives idea of the quality of the clustering result. Note that the Silhouette coefficient only makes sense whenthe number of labels predicted is less than the number of samples clustered.

The advantage of the Silhouette coefficient is that it is bounded between -1 and $+1$. Moreover, it is easy to show that the score is higher when clusters are dense and well separated; a logical feature when speaking about clusters. Furthermore, theSilhouette coefficient is generally higher when clusters are compact.

Taxonomies of Clustering Techniques

Within different clustering algorithms, one can find *soft partition* algorithms, which assign a probability of the data belonging to each cluster, and also *hard partition* algorithms, where each datapoint is assigned precise membership of one cluster. A typical example of a soft partition algorithm is the Mixture of Gaussians [1], which can be viewed as a density estimator method that assigns a confidence or

[2]The intracluster distance of sample *i* is obtained by the distance of the sample to the nearest sample from the same class, and the nearest-cluster distance is given by the distance to the closest samplefrom the cluster nearest to the cluster of sample *i*.

probability to each point in the space. A Gaussian mixture model is a probabilistic model that assumes all the data points are generated from a mixture of a finite number of Gaussian distributions with unknown parameters. The universally used generative unsupervised clustering using a Gaussian mixture model is also known as *EM Clustering*. Each point in the dataset has a soft assignment to the *K* clusters. One can convert this soft probabilistic assignment into membership by picking out the most likely clusters (those with the highest probability of assignment).

An alternative to soft algorithms are the *hard partition* algorithms, which assign a unique cluster value to each element in the feature space. According to the grouping process of the hard partition algorithm, there are two large families of clustering techniques:

- *Partitional algorithms*: these start with a random partition and refine it iteratively. That is why sometimes these algorithms are called "flat" clustering. In this chapter, we will consider two partitional algorithms in detail: K-means and *spectral clus-tering*.
- *Hierarchical algorithms*: these organize the data into hierarchical structures, wheredata can be agglomerated in the bottom-up direction, or split in a top-down manner. In this chapter, we will discuss and illustrate *agglomerative clustering*.

A typical hard partition algorithm is K-means clustering. We will now discuss it in some detail.

K-means Clustering

K-means algorithm is a hard partition algorithm with the goal of assigning each datapoint to a single cluster. K-means algorithm divides a set of *n* samples *X* into *k* disjoint clusters $c_i, i = 1, \ldots, k$, each described by the mean μ_i of the samples in the cluster. The means are commonly called cluster *centroids*. The K-means algorithm assumes that all *k* groups have equal variance.

K-means clustering solves the following minimization problem:

$$\arg\min_c \sum_{j=1}^{k} \sum_{x \in c_j} d(x, \mu_j) = \arg\min_c \sum_{j=1}^{k} \sum_{x \in c_j} ||x - \mu_j||^2 \quad (7.1)$$

where c_i is the set of points that belong to cluster *i* and μ_i is the center of the class c_i. K-means clustering objective function uses the square of the Euclidean distance $d(x, \mu_j) = ||x - \mu_j||^2$, that is also referred to as the *inertia* or *within-cluster sum-of-squares*. This problem is not trivial to solve (in fact, it is NP-hard problem), so the algorithm only hopes to find the global minimum, but may become stuck at a different solution.

In other words, we may wonder whether the centroids should belong to the original set of points:

$$inertia = \sum_{i=0}^{n} \min_{\mu_j \in C}(||x_i - \mu_j||^2)). \quad (7.2)$$

The K-means algorithm, also known as Lloyd's algorithm, is an iterative procedure that searches for a solution of the *K* -means clustering problem and works as follows. First, we need to decide the number of clusters, *k*. Then we apply the following procedure:

1. Initialize (e.g., randomly) the *k* cluster centers, called *centroids*.
2. Decide the class memberships of the *n* data samples by assigning them to thenearest-cluster centroids (e.g., the center of gravity or mean).
3. Re-estimate the *k* cluster centers, c_i, by assuming the memberships found aboveare correct.
4. If none of the *n* objects changes its membership from the last iteration, then exit.Otherwise go to step 2.

Let us illustrate the algorithm in Python. First, we will create three sample distri-butions:

In[5]:

```
MAXN = 40
X = np.concatenate([
    1.25*np.random.randn(MAXN, 2),
    5 + 1.5*np.random.randn(MAXN, 2)])
X = np.concatenate([
    X, [8, 3] + 1.2*np.random.randn(MAXN, 2)])
```

The sample distributions generated are shown in Fig. 7.1 (left). However, the algorithm is not aware of their distribution. Figure 7.1 (right) shows what the algorithm sees. Let us assume that we expect to have three=clusters (*k* 3) and apply the K-means command from the Scikit-learn library:

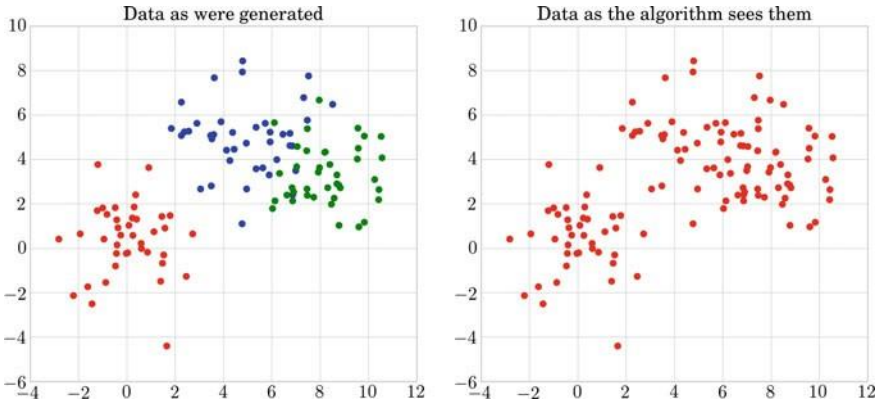


Fig. 7.1 Initial samples as generated (*left*), and samples seen by the algorithm (*right*)

In [6]:
```
from sklearn import cluster

K = 3 # Assuming we have 3 clusters!
clf = cluster.KMeans(init = 'random', n_clusters = K)
clf.fit(X)
```

Out[6]: KMeans(copy_x=True, init='random', max_iter=300,
n_clusters=3, n_init=10, n_jobs=1, precompute_distances=True,random_state=None,
tol=0.0001, verbose=0)

Each clustering algorithm in Scikit-learn is used as follows. First, an object from the clustering technique is instantiated. Then we can use the fitmethod to adjust the learning parameters. We also find the method predict that, given new data, returns the cluster they belong to. For the class, the labels over the training data canbe found in the labels_ attribute or alternatively they can be obtained using the predict method.

How many "mis-clusterings" do we have? In order to see this, we tessellate the space and color all grid points from the same cluster with the same color. Then, weoverlay the initial sample distributions (see Fig. 7.2). In the ideal case, we expect thatin each partitioned subspace the sample points are of the same color. However, as shown in Fig. 7.2, the resulting clustering, which is represented in the figure by the color subspace in gray, does not usually coincide exactly with the initial distribution, which is represented by the color of the data. For example, in the same figure, if most of the blue points belong to the same cluster, there are a few ones that belong to thespace occupied by the green data.

When computing the Rand index, we get:

In [7]:
```
print ('The Adjusted Rand index is: %.2f' %
       metrics.adjusted_rand_score(y.ravel(), clf.labels_))
```

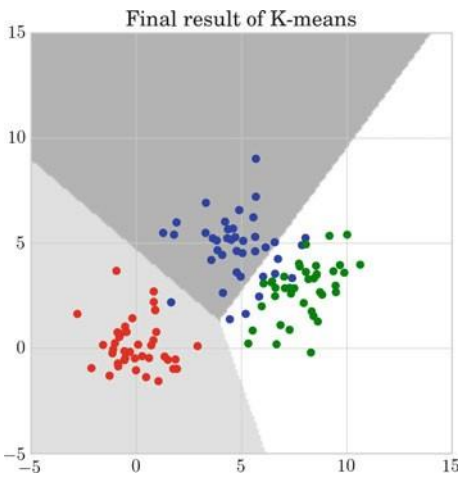


Fig. 7.2 Original samples (*dots*) generated by three distributions and the partitionof the space according to theK-means clustering

Out[7]: The Adjusted Rand index is: 0.66

Taking into account that the Adjusted Rand index belongs to the interval [0, 1], the result of 0.66 in our example means that although most of the clusters were discovered, not 100% of them were; as confirmed by Fig. 7.2.

The inertia can be seen as a measure of how internally coherent the clusters are. Several issues should be taken into account:

- The inertia assumes that clusters are isotropic and convex, since the Euclidean distance is applied, which is isotropic with regard to the different dimensions of the data. However, we cannot expect that the data fulfill this assumption by default. Hence, the K-means algorithm responds poorly to elongated clusters or manifoldswith irregular shapes.
- The algorithm may not ensure convergence to the global minimum. It can be shown that K-means will always converge to a local minimum of the inertia (Eq. (7.2)). It depends on the random initialization of the seeds, but some seeds can result in a poor convergence rate, or convergence to suboptimal clustering. To alleviate the problem of local minima, the K-means computation is often per-formed several times, with different centroid initializations. One way to address this issue is the k-means++initialization scheme, which has been implemented in Scikit-learn (use the init='kmeans++' parameter). This parameter initializes the centroids to be (generally) far from each other, thereby probably leading to better results than random initialization.
- This algorithm requires the number of clusters to be specified. Different heuristics can be applied to predetermine the number of seeds of the algorithm.
- It scales well to a large number of samples and has been used across a large rangeof application areas in many different fields.

In summary, we can conclude that K-means has the advantages of allowing the easy use of heuristics to select good seeds; initialization of seeds by other methods;multiple points to be tried. However, in contrast, it still cannot ensure that the localminima problem is overcome; it is iterative and hence slow when there are a lot of high-dimensional samples; and it tends to look for spherical clusters.

Spectral Clustering

Up to this point, the clustering procedure has been considered as a way to find datagroups following a notion of compactness. Another way of looking at what a clusteris is provided by connectivity (or similarity). Spectral clustering [2] refers to a familyof methods that use spectral techniques. Specifically, these techniques are related tothe eigendecomposition of an affinity or similarity matrix and solve the problem ofclustering according to the connectivity of the data. Let us consider an ideal similarity matrix of two clear sets.

Let us denote the similarity matrix, S, as the matrix $S_{ij} = s(x_i, x_j)$ which gives the similarity between observations x_i and x_j. Remember that we can model similarity

using the Euclidean distance, $d(x_i, x_j) = ||x_i - x_j||^2$, by means of a Gaussian Kernel as follows:

$$s(x_i, x_j) = exp(-\alpha||x_i - x_j||^2),$$

where α is a parameter. We expect two points from different clusters to be far away from each other. However, if there is a sequence of points within the cluster that forms a "path" between them, this also would lead to big distance among some of the points from the same cluster. Hence, we define an affinity matrix A based on the similarity matrix S, where A contains positive values and is symmetric. This can be done, for example, by applying a k-nearest neighbor that builds a graph connecting just the k closest data points. The symmetry comes from the fact that A_{ij} and A_{ji} give the distance between the same points. Considering the affinity matrix, the clustering can be seen as a graph partition problem, where connected graph components correspond to clusters. The graph obtained by spectral clustering will be partitioned so that graph edges connecting different clusters have low weights, and vice versa. Furthermore, we define a degree matrix D, where each diagonal value is the degree of the respective graph node and all other elements are 0. Finally, we can compute the unnormalized graph Laplacian (U D A) and/or a normalized version of the Laplacian (L), as follows:

- *Simple Laplacian*: $L = D^{-1}A$, which corresponds to a random walk, being D^{-1} the transition matrix. Spectral clustering obtains groups of nodes such that the random walk corresponds to seldom transitions from one group to another.
- *Normalized Laplacian*: $L = D^{-1} U D^{-1}$.
- *Generalized Laplacian*: $L = D^{-1}U$.

If we assume that there are k clusters, the next step is to find the k small-est eigenvectors, without considering the trivial constant eigenvector. Each row of the matrix formed by the k smallest eigenvectors of the Laplacian matrix defines a transformation of the data x_i. Thus, in this transformed space, we can apply K-means clustering in order to find the final clusters. If we do not know in advance the number of clusters, k, we can look for sudden changes in the sorted eigenvalues of the matrix, U, and keep the smallest ones.

Hierarchical Clustering

Another well-known clustering technique of particular interest is hierarchical cluster-ing. Hierarchical clustering is comprised of a general family of clustering algorithms that construct nested clusters by successive merging or splitting of data. The hier-archy of clusters is represented as a tree. The tree is usually called a *dendrogram*. The root of the dendrogram is the single cluster that contains all the samples; the leaves are the clusters containing only one sample each. This is a nice tool, since it can be straightforwardly interpreted: it "explains" how clusters are formed and visualizes clusters at different scales. The tree that results from the technique shows

the similarity between the samples. Partitioning is computed by selecting a cut onthe tree at a certain level.

In general, there are two types of hierarchical clustering:

- *Top-down* divisive clustering applies the following algorithm:
  - Start with all the data in a single cluster.
  - Consider every possible way to divide the cluster into two.
  - Choose the best division.
  - Recursively, it operates on both sides until a stopping criterion is met. That can be something as follows: there are as much clusters as data; the predetermined number of clusters has been reached; the maximum distance between all possible partition divisions is smaller than a predetermined threshold; etc.

- *Bottom-up* agglomerative clustering applies the following algorithm:
  - Start with each data point in a separate cluster.
  - Repeatedly join the closest pair of clusters.
  - At each step, a stopping criterion is checked: there is only one cluster; a prede-termined number of clusters has been reached; the distance between the closestclusters is greater than a predetermined threshold; etc.

This process of merging forms a binary tree or hierarchy.

When merging two clusters, a question naturally arises: How to measure the similarity of two clusters? There are different ways to define this with different results for the agglomerative clustering. The linkage criterion determines the metricused for the cluster merging strategy:

- *Maximum* or *complete* linkage minimizes the maximum distance between observations of pairs of clusters. Based on the similarity of the two least similar membersof the clusters, this clustering tends to give tight spherical clusters as a final result.*Average* linkage averages similarity between members, i.e., minimizes the averageof the distances between all observations of pairs of clusters.
- *Ward* linkage minimizes the sum of squared differences within all clusters. It is thus a variance-minimizing approach and in this sense is similar to the K-means objective function, but tackled with an agglomerative hierarchical approach.

Let us illustrate how the different linkages work with an example. Let us generatethree clusters as follows:

In [8]:

```
MAXN1 = 500
MAXN2 = 400
MAXN3 = 300
X1 = np.concatenate([
    2.25 * np.random.randn(MAXN1, 2),
    4 + 1.7*np.random.randn(MAXN2, 2)])
X1 = np.concatenate([
    X1, [8, 3] + 1.9*np.random.randn(MAXN3, 2)])

y1 = np.concatenate([
    np.ones((MAXN1, 1)),
    2 * np.ones((MAXN2, 1))])
y1 = np.concatenate([
    y1, 3 * np.ones((MAXN3, 1))]).ravel()
y1 = np.int_(y1)
labels_y1 = ['+', '*', 'o']
colors = ['r', 'g', 'b']
```

Let us apply agglomerative clustering using the different linkages:

In [9]:

```
from sklearn.cluster import AgglomerativeClustering

for linkage in ('ward', 'complete', 'average'):
    clustering = AgglomerativeClustering(linkage = linkage,
                                         n_clusters =3)
    clustering.fit(X1)

    x_min , x_max = np.min (X1, axis = 0) , np.max (X1,axis
        = 0)
    X1 = (X1 - x_min ) / ( x_max - x_min )
    plt.figure ( figsize =(5 , 5))
    for i in range (X1.shape [0]) :
        plt.text(X1[i, 0], X1[i, 1], labels_y1[y1[i]-1],
                 color = colors[y1[i]-1])
    plt.title("\%s linkage " \% linkage, size = 20)
    plt.tight_layout()

plt.show()
```

The results of the agglomerative clustering using the different linkages: complete, average, and Ward are given in Fig. 7.3. Note that agglomerative clustering exhibits “rich get richer” behavior that can sometimes lead to uneven cluster sizes, with average linkage being the worst strategy in this respect and Ward linkage giving themost regular sizes. Ward linkage is an attempt to form clusters that are as compactas possible, since it considers inter- and intra-distances of the clusters. Meanwhile, for non-Euclidean metrics, average linkage is a good alternative. Average linkage can produce very unbalanced clusters, it can even separate a single data point into aseparate cluster. This fact would be useful if we want to detect outliers, but it may be undesirable when two clusters are very close to each other, since it would tend tomerge them.

Agglomerative clustering can scale to a large number of samples when it is usedjointly with a connectivity matrix, but it is computationally expensive when no con-

nectivity constraints are added between samples: it considers all the possible mergesat each step.

Adding Connectivity Constraints

Sometimes, we are interested in introducing a connectivity constraint into the clus- tering process so that merging of nonadjacent points is avoided. This can be achieved by constructing a connectivity matrix that defines which are the neighboring samplesin the dataset. For instance, in the example in Fig. 7.4, we want to avoid the forma-tion of clusters of samples from the different circles. A sample code to compute agglomerative clustering with connectivity would be as follows:

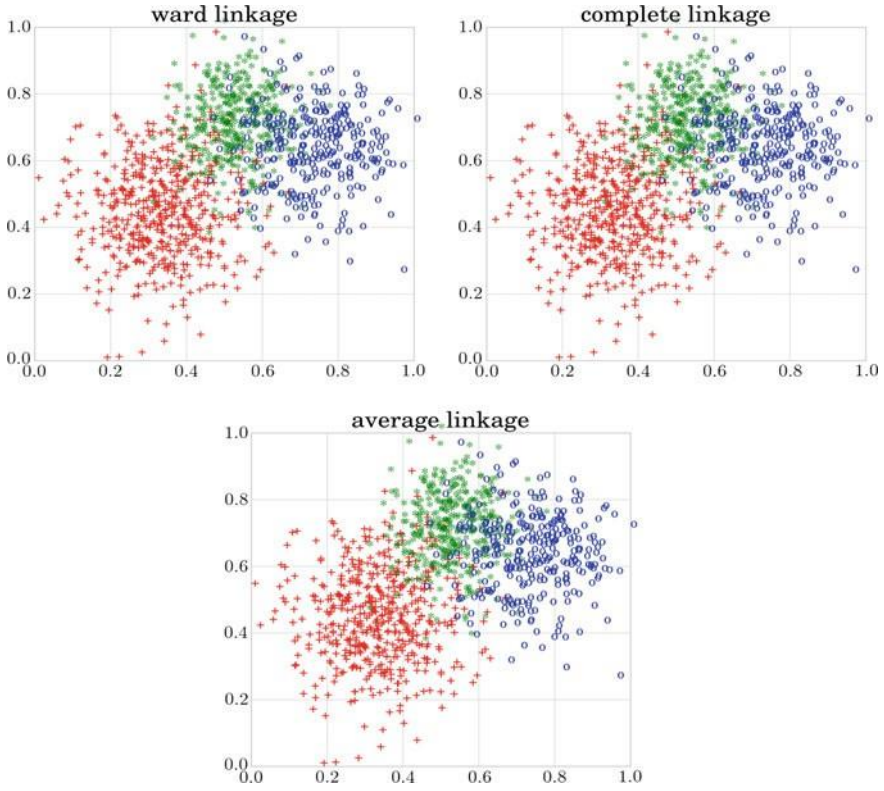


Fig. 7.3 Illustration of agglomerative clustering using different linkages: Ward, complete, and average. The symbol of each data point corresponds to the original class generated and the color corresponds to the cluster obtained

Fig. 7.4 Illustration of agglomerative clustering without (*top row*) and with (*bottom row*) a connectivity graph using the three linkages (from *left* to *right*): average, complete, and Ward. The *colors*correspond to the clusters obtained

In [10]:

```
connectivity = kneighbors_graph(X, 30)
model = AgglomerativeClustering(linkage = 'average',
    connectivity = connectivity, n_clusters = 8)
model.fit(X)
```

A connectivity constraint is useful to impose a certain local structure, but it also makes the algorithm faster, especially when the number of the samples is large. A connectivity constraint is imposed via a *connectivity matrix*: a sparse matrix that only has elements at the intersection of a row and a column with indexes of the dataset that should be connected. This matrix can be constructed from a priori information or can be learned from the data, for instance using kneighbors_graph to restrict merging to nearest neighbors or using image.grid_to_graphto limit mergingto neighboring pixels in an image, both from Scikit-learn. This phenomenon can beobserved in Fig. 7.4, where in the first row we see the results of the agglomerative clustering without using a connectivity graph. The clustering can join data from different circles (e.g., the black cluster). At the bottom, the three linkages use a connectivity graph and thus two of them avoid joining data points that belong to different circles (except the Ward linkage that attempts to form compact and isotropic clusters).

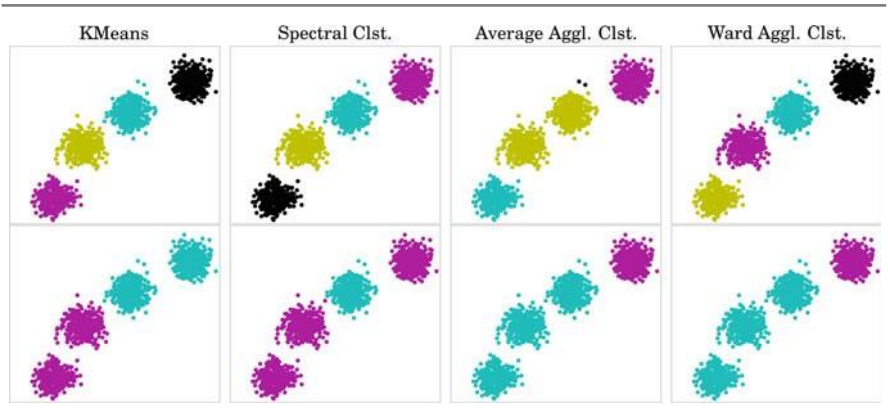


Fig. 7.5 Comparison of the different clustering techniques (from *left* to *right*): K-means, spectral clustering, and agglomerative clustering with average and Ward linkage on simple compact datasets. In the *first row*, the expected number of clusters is $k = 2$ and in the *second row*: $k = 4$

Comparison of Different Hard Partition Clustering Algorithms Let us compare the behavior of the different clustering algorithms discussed so far.For this purpose, we generate three different datasets' configurations:

(a) 4 spherical groups of data;
(b) a uniform data distribution; and
(c) a non-flat configuration of data composed of two moon-like groups of data.

An easy way to generate these datasets is by using Scikit-learn that has predefined functions for it: datasets.make_blobs(), datasets.ma- ke_ moons(), etc.

We apply the clustering techniques discussed above, namely K-means, agglom-erative clustering with average linkage, agglomerative clustering with Ward linkage, and spectral clustering. Let us test the behavior of the different algorithms assuming $k = 2$ and $k = 4$. Connectivity is applied in the algorithms where it is applicable.

In the simple case of separated clusters of data and $k = 4$, most of the clustering algorithms perform well, as expected (see Fig. 7.5). The only algorithm that could not discover the four groups of samples is the average agglomerative clustering. Since it allows highly unbalanced clusters, the two noisy data points that are quite separated from the closest two blobs were considered as a different cluster, while the two central blobs were merged in one cluster. In case of $k = 2$, each of the methods is obligated to join at least two blobs in a cluster.

Regarding the uniform distribution of data (see Fig. 7.6), K-means, Ward linkage agglomerative clustering and spectral clustering tend to yield even and compact clusters; while the average linkage agglomerative clustering attempts to join close points as much as possible following the "rich get richer" rule. This results in a

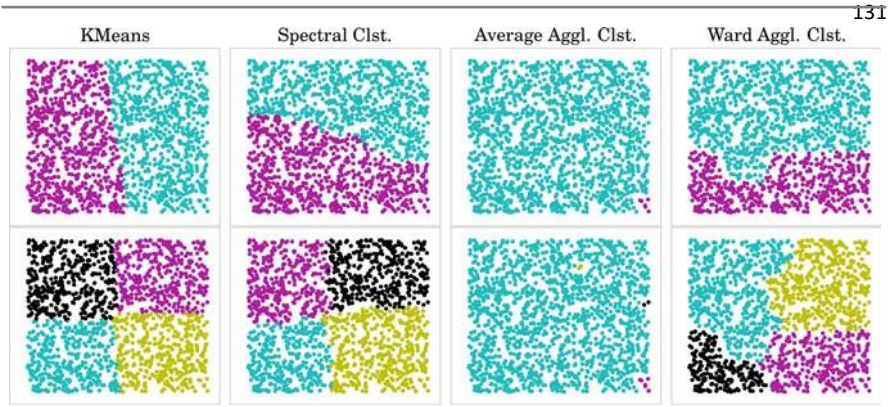


Fig. 7.6 Comparison of the different clustering techniques (from *left* to *right*): K-means, spectral clustering, and agglomerative clustering with average and Ward linkage on uniformly distributeddata. In the *first row*, the number of clusters assumed is $k = 2$ and in the *second row*: $k = 4$

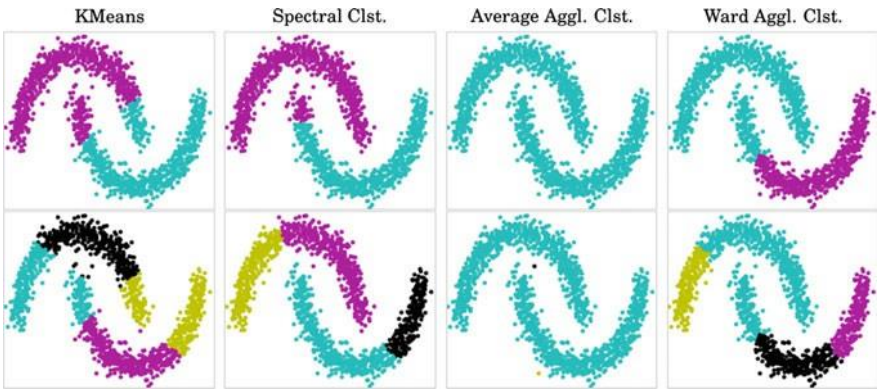


Fig. 7.7 Comparison of the different clustering techniques (from *left* to *right*): K-means, spectral clustering, and agglomerative clustering with average and Ward linkage on non-flat geometry datasets. In the *first row*, the expected number of clusters is $k = 2$ and in the *second row*: $k = 4$

second cluster of a small set of data. This behavior is observed in both cases: $k = 2$ and $k = 4$.

Regarding datasets with more complex geometry, like in the moon dataset (see Fig. 7.7), K-means and Ward linkage agglomerative clustering attempt to construct compact clusters and thus cannot separate the moons. Due to the connectivity con- straint, the spectral clustering and the average linkage agglomerative clustering sep-arated both moons in case of $k = 2$, while in case of $k = 4$, the average linkage agglomerative clustering clustered most of datasets correctly separating some of thenoisy data points as two separate single clusters. In the case of spectral clustering, looking for four clusters, the method splits each of the two moon datasets into two clusters.

	0	1	2	3	4	5	6	7	8	9	10	11
GEO												
Albania	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Austria	0.52	5.25	9.98	0.64	1.22	0.61	1.01	2.64	1.63	5.89	5.90	11.20
Belgium	0.34	6.25	11.90	0.32	0.61	0.78	1.54	2.79	1.46	6.57	6.44	12.51
Bulgaria	0.63	3.35	8.96	0.74	1.99	0.92	0.80	1.76	0.61	4.10	4.18	10.95
Croatia	0.26	4.24	NaN	0.03	NaN	0.65	1.87	0.97	0.78	4.27	4.42	NaN

Fig. 7.8 Expenditure on different educational indicators for the first five countries in the Eurostat dataset

Case Study

In order to illustrate clustering with a real dataset, we will now analyze the indicators of spending on education among the European Union member states, provided by the Eurostat data bank.[3] The data are organized by year (TIME) from 2002 until 2011 and country (GEO): ('Albania', 'Austria', 'Belgium', 'Bulgaria', etc.). Twelve indicators (INDIC_ED) of financing of education with their corresponding values (Value) are given: (1) Expenditure on educational institutions from private sources as % of gross domestic product (GDP), for all levels of education combined; (2) Expenditure on educational institutions from public sources as % of GDP, for all levels of government combined, (3) Expenditure on educational institutions from public sources as % of total public expenditure, for all levels of education combined,
(4) Public subsidies to the private sector as % of GDP, for all levels of education combined, (5) Public subsidies to the private sector as % of total public expenditure, for all levels of education combined, etc. We can store the 12 indicators for a given year (e.g., 2010) in a table. Figure 7.8 provides visualization of the first five countries in the table.

As we can observe, this is not a clean dataset, since there are values missing. Some countries have very limited information and should be excluded. Other countries may still not collect or have access to a few indicators. For these last cases, we can proceed in two ways: (a) fill in the gaps with some non-informative, non-biasing data; or (b) drop the features with missing values for the analysis. If we have many features and only a few have missing values, then it is not very harmful to drop them. However, if missing values are spread across most of the features, we eventually have to deal with them. In our case, both options seem reasonable, as long as the number of missing features for a country is not too large. We will proceed in both ways at the same time. We apply both options: filling the gap with the mean value of the feature and the dropping option, ignoring the indicators with missing values. Let us now apply K-means clustering to these data in order to partition the countries according to

.

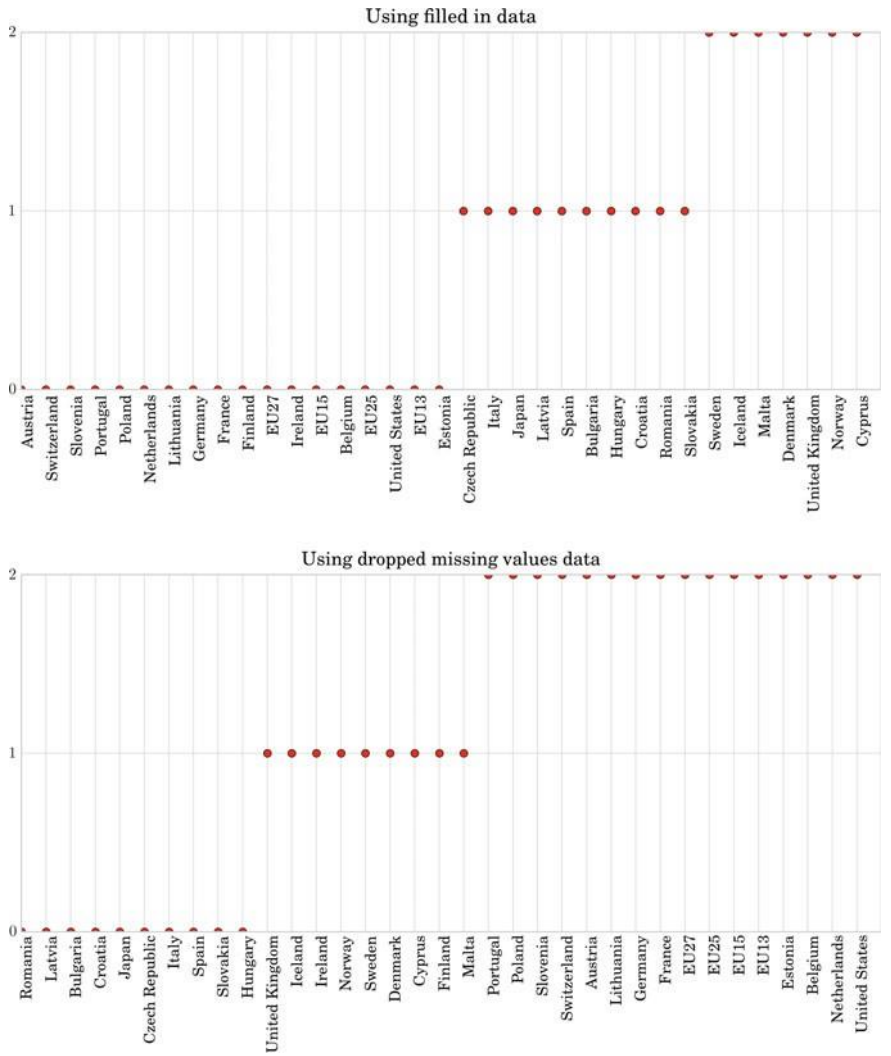


Fig. 7.9 Clustering of the countries according to their educational expenditure using filled-in (*top row*) and dropped (*bottom row*) missing values

their investment in education and check their profiles. Figure 7.9 shows the results of this K-means clustering. We have sorted the data for better visualization. Ata simple glance, we can see that the partitions (top and bottom of Fig. 7.9) are different. Most countries in cluster 2 in the filled-in dataset correspond to cluster 0in the dropped missing values dataset. Analogously, most of cluster 0 in the filled-in dataset correspond to cluster 1 in the dropped missing values dataset; and most countries from cluster 1 in the filled-in dataset correspond to cluster 2 in the dropped

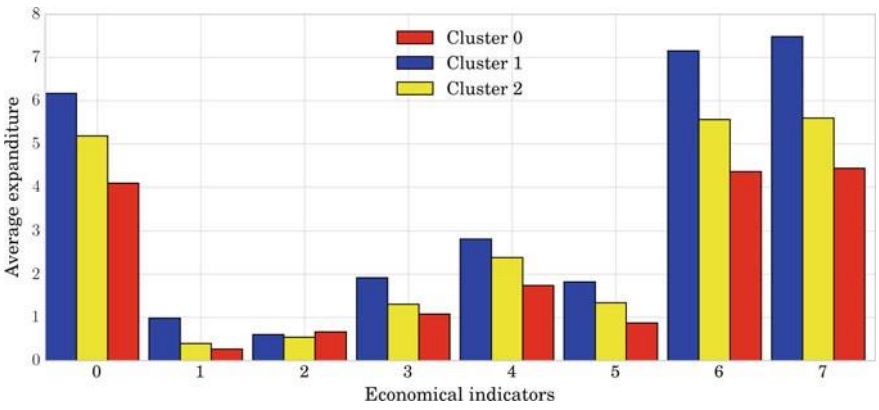


Fig. 7.10 Mean expenditure of the different clusters according to the 8 indicators of the indicators-dropped dataset

set. Still, there are some countries that do not follow this rule. That is, looking at both clusterings, they may yield similar (up to label permutation) results, but theywill not necessarily always coincide. This is mainly due to two aspects: the randominitialization of the K-means clustering and the fact that each method works in adifferent space (i.e., dropped data in 8D space *vs* filled-in data, working in 12Dspace). Note that we should not consider the assigned absolute cluster value, sinceit is irrelevant. The mean expenditure of the different clusters is shown by differentcolors according to the 8 indicators of the indicators-dropped dataset (see Fig. 7.10).So, without loss of generality, we continue analyzing the set obtained by dropping missing values. Let us now check the clusters and check their profile by looking atthe centroids. Visualizing the eight values of the three clusters (see Fig. 7.10), we cansee that cluster 1 spends more on education for the 8 educational indicators, while cluster 0 is the one with least resources invested in education.

Let us consider a specific country, e.g., Spain and its expenditure on education. If we refine cluster 0 further and check how close members are from this cluster to cluster 1, it may give us a hint as to a possible ordering. When visualizing the distance to cluster 0 and 1, we can observe that Spain, while being from cluster 0, hasa smaller distance to cluster 1 (see Fig. 7.11). This should make us realize that using 3 clusters probably does not sufficiently represent the groups of countries. So we redothe process, but applying $k = 4$: we obtain 4 clusters. This time cluster 0 includes the EU members with medium expenditure (Fig. 7.12). This reinforce the intuition about Spain being a limit case in the former clustering. The clusters obtained are asfollows:

- Cluster 0: (‘Austria’, ‘Estonia’, ‘EU13’, ‘EU15’, ‘EU25’, ‘EU27’, ‘France’, ‘Germany’, ‘Hungary’, ‘Latvia’, ‘Lithuania’, ‘Netherlands’, ‘Poland’, ‘Portugal’, ‘Slovenia’, ‘Spain’, ‘Switzerland’, ‘United Kingdom’, ‘United States’)

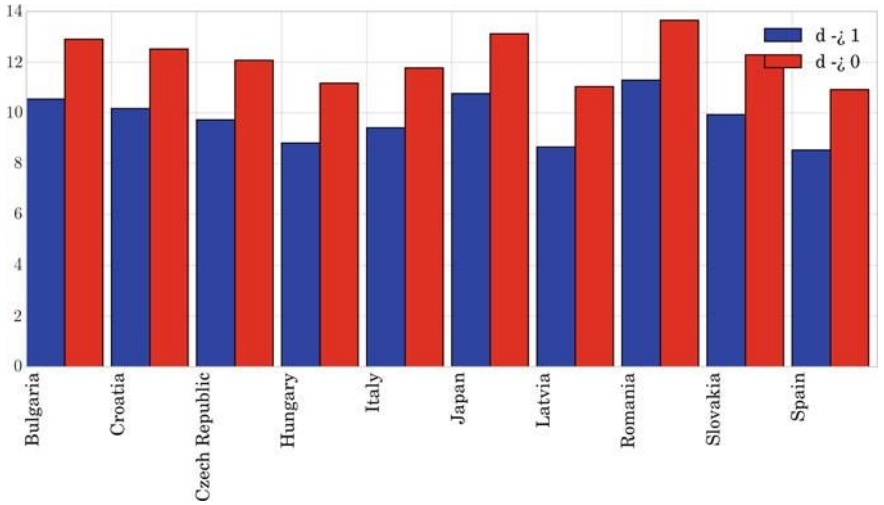


Fig. 7.11 Distance of countries in cluster 0 to centroids of cluster 0 (in *red*) and cluster 1 (in *blue*)

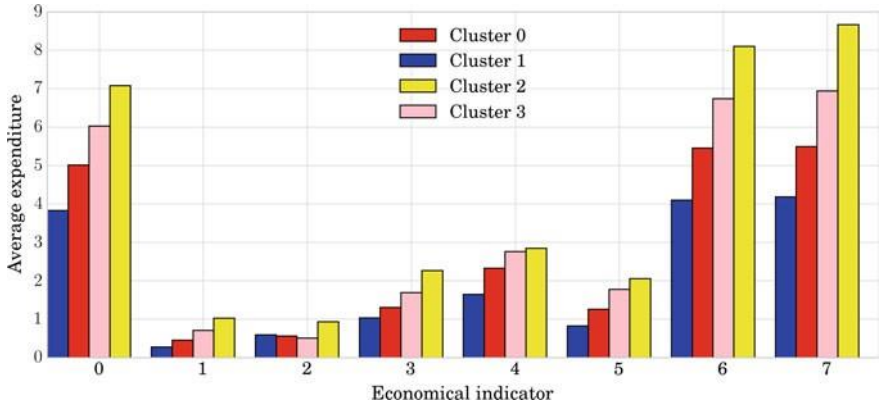


Fig. 7.12 K-means applied to the Eurostat dataset grouping the countries into four clusters

- Cluster 1: ('Bulgaria', 'Croatia', 'Czech Republic', 'Italy', 'Japan', 'Romania','Slovakia')
- Cluster 2: ('Cyprus', 'Denmark', 'Iceland')
- Cluster 3: ('Belgium', 'Finland', 'Ireland', 'Malta', 'Norway', 'Sweden')

We can repeat the process using the alternative clustering techniques and compare their results. Let us first apply spectral clustering. The corresponding code will be as follows:

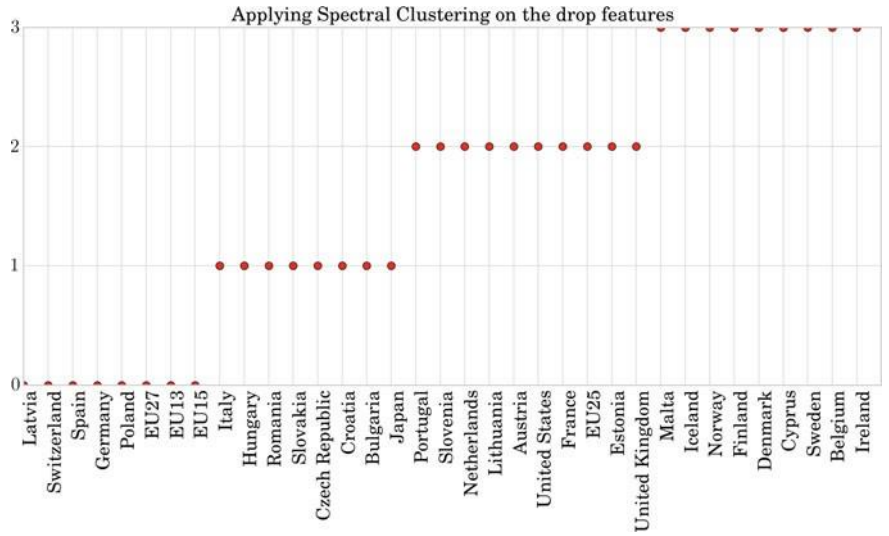


Fig. 7.13 Spectral clustering applied to the European countries according to their expenditure on education

In [11]:

```
X = StandardScaler().fit_transform(edudrop.values)
distances = euclidean_distances(edudrop.values)
spectral = cluster.SpectralClustering(
    n_clusters = 4, affinity = "nearest_neighbors")
spectral.fit(edudrop.values)
y_pred = spectral.labels_.astype(np.int)
```

The result of this spectral clustering is shown in Fig. 7.13. Note that in general, the aim of spectral clustering is to obtain more balanced clusters. In this way, the predicted cluster 1 merges clusters 2 and 3 of the K-means clustering, cluster 2 corresponds to cluster 1 of the K-means clustering, cluster 0 mainly shifts to cluster2, and cluster 3 corresponds to cluster 0 of the K-means.

Applying agglomerative clustering, not only we do obtain different clusters, but also we can see how different clusters are obtained. Thus, in some way it is giving us information on which the most similar pairs of countries and clusters are. The corresponding code that applies the agglomerative clustering will be as follows:

In [12]:

```
from scipy.cluster.hierarchy import linkage, dendrogram
from scipy.spatial.distance import pdist

X_train = edudrop.values
dist = pdist(X_train, 'euclidean')
linkage_matrix = linkage(dist, method = 'complete');

plt.figure(figsize = (11.3, 11.3))
dendrogram(linkage_matrix, orientation="right",
           color_threshold = 3,
           labels = wrk_countries_names,
           leaf_font_size = 20);
plt.tight_layout()
```

In Scikit-learn, the parameter *color_threshold* of the command dendro- gram() colors all the descendent links below a cluster node k the same color if k is the first node below the color_threshold. All links connecting nodes with distances greater than or equal to the threshold are colored blue. Hence, using *color_threshold*
= 3, the clusters obtained are as follows:

- Cluster 0: ('Cyprus', 'Denmark', 'Iceland')
- Cluster 1: ('Bulgaria', 'Croatia', 'Czech Republic', 'Italy', 'Japan', 'Romania', 'Slovakia')
- Cluster 2: ('Belgium', 'Finland', 'Ireland', 'Malta', 'Norway', 'Sweden')
- Cluster 3: ('Austria', 'Estonia', 'EU13', 'EU15', 'EU25', 'EU27', 'France', 'Germany', 'Hungary', 'Latvia', 'Lithuania', 'Netherlands', 'Poland', 'Portugal', 'Slovenia', 'Spain', 'Switzerland', 'United Kingdom', 'United States')
  Note that, to a high degree, they correspond to the clusters obtained by the K-means (except for permutation of cluster labels, which is irrelevant).

Figure 7.14 shows the construction of the clusters using complete linkage agglomerative clustering. Different cuts at different levels of the dendrogram allow us to obtain different numbers of clusters.

To summarize, we can compare the results of the three clustering approaches. We cannot expect the results to coincide, since the different approaches are based on different criteria for constructing clusters. Nonetheless, we can still observe that in this case, K-means and the agglomerative approaches gave the same results (up to apermutation of the number of cluster, which is irrelevant); while spectral clustering gave more evenly distributed clusters. This later approach fused clusters 0 and 2 ofthe agglomerative clustering in cluster 1, and split cluster 3 of the agglomerative clustering into its clusters 0 and 3. Note that these results could change when usingdifferent distances among data.

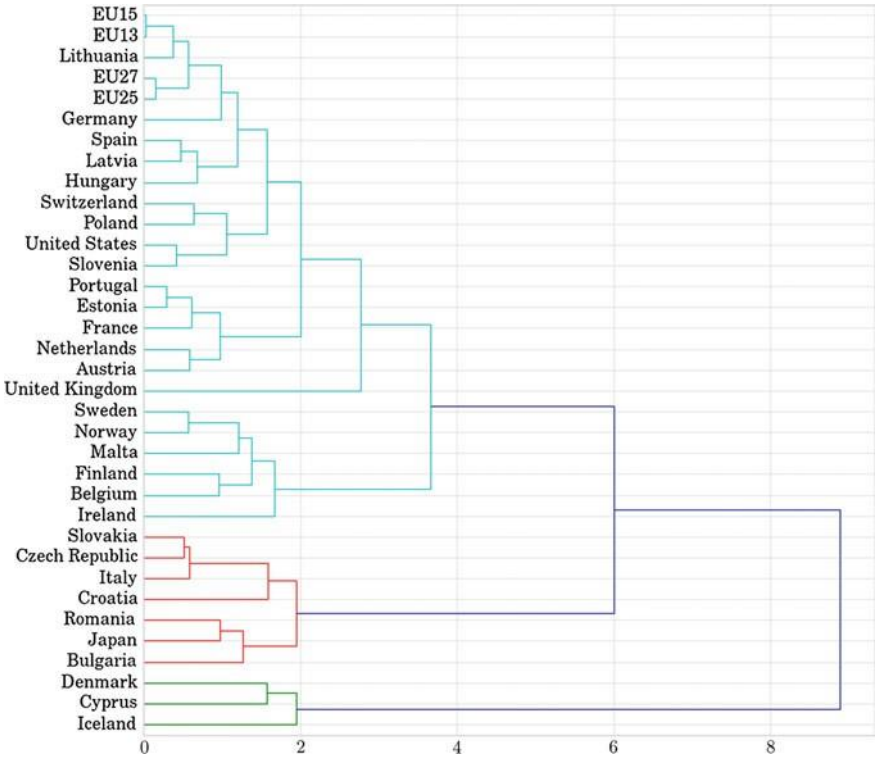


Fig. 7.14 Agglomerative clustering applied to cluster European countries according to their expenditure on education